

Sliding Puzzle Project — Development Log

Day 4 — Saturday, August 8, 2026

Goal for the project

Build four Python puzzle projects (a playable sliding tile puzzle, a solver for it, a playable 3D Rubik's cube, and a solver for that) as a learning exercise. The user writes all code themselves; guidance is conceptual, not code handed over directly.

Today's focus

Stage 3 (shuffling) completed and verified. Stage 4 (rendering) begun and largely completed: first use of pygame, opening a window sized to the board, and drawing a static shuffled board as coloured tiles with centred-in-progress numbers.

What happened

- Finished `shuffle.py`: `shuffle(size)` builds a solved board, then makes 200 random legal moves, excluding the immediate reverse of the previous move each time by removing the just-vacated blank position from the next candidate list before choosing. Confirmed correct by hand-tracing (the just-vacated position is always adjacent to the new blank position, so `.remove()` can never fail) and by running it — output showed a genuinely scrambled board with `is_solved()` correctly returning `False`.
- Discussed expected solve length for context: a 3x3 has a maximum optimal solution of 31 moves (average ~21-22); a 4x4 has a maximum of 80 moves (average ~52-53) — 200 shuffle moves is more than enough to reach a genuinely random state for either size, so the difficulty is expected and not a sign of a shuffle bug.
- Started `render.py`, importing `pygame`, `board`, `movement_logic`, and `shuffle`. Established that `board.py`'s `num_of_rows/num_of_columns` work directly on a dictionary (not just a flat list), since they only ever call `len()` internally — no conversion needed between the two representations.
- Built the pygame window sized exactly to the board: `tile_size` (100px) multiplied by column/row counts gives window width/height, passed to `pygame.display.set_mode()` as a single (width, height) tuple.
- Built the standard pygame game loop: check for the QUIT event to allow the window to close, fill the background each frame, then `flip()` to actually display what was drawn.
- Wrote the tile-drawing loop: iterating `dictionary.items()`, skipping the blank (" "), converting each (row, col) into a pixel position via `x = col * tile_size`, `y = row * tile_size`, and drawing a coloured square with `pygame.draw.rect()`. Caught and fixed two real bugs along the way (see Concepts section).
- Added text rendering: created a font object once (`pygame.font.Font(None, 50)`), then inside the loop rendered each tile's number as an image with `font.render()` and drew it onto the screen with `screen.blit()`. Confirmed numbers now appear on the tiles, though not yet centred within each square — flagged as a follow-up refinement, along with a visual cross-check against `print_grid()` output.

Concepts & Lessons Learned (study notes)

`pygame.init()`

Starts up all of pygame's internal subsystems at once (display, input, fonts, timers, etc.) before any of them can be used. Must be the very first pygame-related call in a program; skipping it can cause later pygame calls to fail or behave unpredictably.

Why a game needs a loop at all

A plain script runs top to bottom once and exits. A window that needs to stay open, keep responding to input, and keep redrawing itself many times a second requires an explicit loop (while running;) that keeps repeating until something (like clicking the close button) sets a flag to stop it.

`pygame.display.set_mode((width, height))`

Opens an actual window of the given pixel size and returns a Surface object (conventionally named `screen`) representing that window's drawable canvas. Note the nested parentheses: the function takes exactly one argument, which is itself a (width, height) tuple — writing `set_mode(width, height)` without the inner tuple is a `TypeError`, since the function expects one argument, not two.

`pygame.event.get()` and `pygame.QUIT`

`pygame` silently collects everything that happened since the last check (key presses, clicks, the window being closed) into a queue; `event.get()` retrieves that list so the program can react to it. Each event has a `.type`; `pygame.QUIT` specifically is the built-in label for "the user clicked the window's close button." Checking for it and setting `running = False` is what allows the window to actually close — without this check, clicking the X would have no effect.

`screen.fill()` and `pygame.display.flip()`

`fill()` paints the whole window one solid colour (an RGB tuple, 0-255 per channel) — done once per frame to clear whatever was drawn last frame before redrawing. Critically, drawing commands in `pygame` (`fill`, `draw.rect`, `blit`) only affect an internal, invisible buffer — nothing becomes visible on screen until `display.flip()` is called, which pushes that buffer to the actual window. Both need to happen every frame, inside the loop.

`pygame`'s pixel coordinate convention

(0,0) is the top-left corner of the window, not the bottom-left. X increases to the right (as expected), but Y increases DOWNWARD — the opposite of a standard maths graph. This matters directly for mapping board (row, col) to screen (x, y): since row increases downward on the puzzle (top row = row 0) and `pygame`'s y also increases downward, `x = col * tile_size` and `y = row * tile_size` line up correctly with no adjustment needed, as long as row and col are consistently mapped to y and x respectively and never swapped (a pitfall flagged in the original project plan).

Describing a rectangle: (x, y, width, height)

A single (x, y) point only describes a position, not a size — `pygame.draw.rect()` needs all four values: the (x, y) top-left corner of the rectangle, plus its width and height. For a square puzzle tile, width and height are both just `tile_size`.

`pygame.draw.rect(surface, color, rect)` — function call vs. assignment bug

A real bug caught during this session: writing `pygame.draw.rect = (screen, color, rectangle)` uses `=` (assignment) instead of `()` (a function call). This doesn't draw anything — it silently overwrites the name `pygame.draw.rect` with a plain tuple, destroying the function itself for the rest of the program. The fix is calling the function with parentheses: `pygame.draw.rect(screen, color, rectangle)`. General lesson: `=` assigns a value to a name; `()` calls a function using the values inside — mixing them up doesn't raise an error in this case, it just quietly does something completely different from what was intended.

Drawing code has to live inside the frame loop, not after it

A second real bug: the tile-drawing loop was initially written after the `while running:` loop had already been defined, at the same indentation level rather than nested inside it. Since `while running:` only exits once the window is closed, code written after it (at the same indentation) never runs while the window is actually open — it would only execute once the user had already closed the window, at which point there's nothing left to draw on. Anything that needs to appear on screen while the window is open must be indented inside the while loop, in the correct order: clear background, draw everything, then `flip()`.

Text isn't drawn directly — it's rendered to an image, then blitted

Unlike shapes (drawn directly with `pygame.draw.rect`), text must first be converted into an image (a Surface) using `font.render(text, antialias, color)`, then copied onto the screen with `screen.blit(image, position)`. `font.render()` requires a real string, hence wrapping numeric tile values in `str(piece)` before rendering. "Blit" is a standard graphics-programming term meaning "copy this image onto that surface at this position." The font object itself (`pygame.font.Font(None, size)`) only needs to be created once, outside the frame loop, since the same font is reused for every tile every frame — but `font.render()` must be called fresh inside the loop each time, since the text (piece) changes from tile to tile.

Status at end of day

Stage 3 (shuffling) is complete and verified. Stage 4 (rendering) is functionally working: a shuffled board opens in a correctly sized pygame window, showing coloured tiles with their numbers drawn on top, refreshing every frame, with the window closing cleanly on request. Remaining before Stage 4 is fully signed off: centre the numbers within each tile (currently anchored to the tile's top-left corner), and do a visual cross-check against `print_grid()` output to confirm no row/column mix-up.

Next up

Finish centring tile numbers, do the row/column visual sanity check, then move to Stage 5 — input handling: arrow keys (via `KEYDOWN`) and mouse clicks (converted from pixel position back to (row, col)), both routed through the existing `apply_move()` logic rather than duplicating movement rules in the rendering code.